

Pointers and Arrays: 1-D Arrays

Similarities and difference (Pointers and Arrays)

Dynamic Arrays 1-D Array (Runtime)

→ Allocation → take size as input from user/read it from a file.

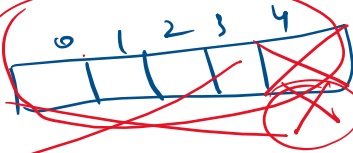
→ Deallocation → ① Data (Same as Static Arrays)

→ Processing → ② Resize whole Array.

```
int size = 5;  
int *arr = new int[size];  
[delete arr[4];  
[delete [] arr;
```

block ^{life time}
Allocated for array
cannot be changed (large/extended shrink it)

20 bytes = 5x4



size = size - 1;

size = 4

48 size

1) if we change size variable array's size will also change.

2) deallocate any element

Resize.

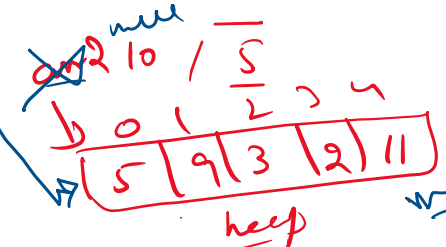
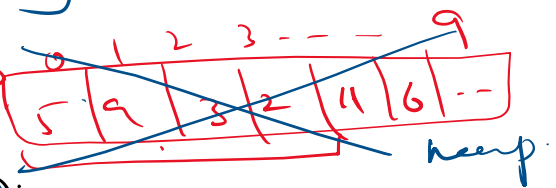
- 1) Create new array of (smaller or larger size)
- 2) copy data from old array to new one element by element
- 3) deallocate original old array.

4) update the pointer variable.

5) update the size

```
void Shrink (int* arr, int size) {  
    int size = 10;  
    int *arr = new int[size];  
    for (int i = 0; i < size; i++)  
        cin >> arr[i];  
    Shrink (int* arr, int size);  
}
```

arr



memory leak

① int *arr2 = new int [size/2];

② for (int i = 0; i < size/2; i++)
 arr2[i] = arr[i]; ← copy

③ delete [] arr;

④ arr = arr2;
 arr2 = nullptr;

arr = arr2;
delete [] arr;

⑤ size = size / 2;

function and dynamic 1D Arrays → exist between function

→ pass to functions (reference, value) ^{pointer (base)}

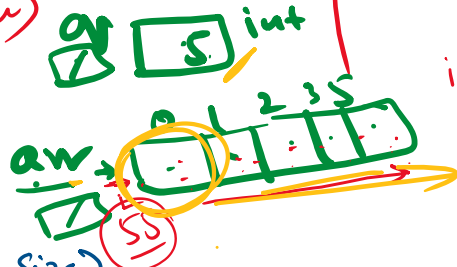
→ Can return Address of array from function.

→ Data is always passed by reference through pointers.

→ Allocate ^{pass by reference}
 → Input ^{by value}
 → print Data ^{by value, const}
 → processing ^{by value}
 → deallocation ^{by value}
 → Extend ^{by value (reference)}
 → Shrink ^{reference}

void fun(int *p);

void fun2(int *p);



single variable
 Array

Static dynamic

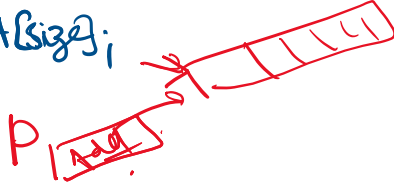
int main()

```
int *q = new int(5);
fun(q); fun2(q);
int *arr = new int[*q];
fun(arr);
fun2(arr);
```

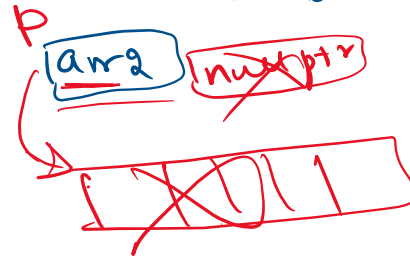
void allocate(int *p, int size)

{ p = new int[size];

}



```
int *arr2 = nullptr;
allocate(arr2, 10);
```



arr2 = allocate(5);

int * allocate (int size)

```
{ int *p;
  p = new int[size];
```

```
  return p;
```

int * p = return and remove odd (arr2, 10);

void Input(int *p, int size)

```
{ for (int i = 0; i < size; i++)
```

```
  cin >> p[i];
```

read write data

delete [] p;

void print(const int *p, int size)

```
{ for loop
```

```
  cout << p[i];
```

read data

→ deallocation(arr2);

free

void deallocation(int *p)

```
{ delete [] p;
```

```
  p = nullptr;
```

}

Shallow vs. deep copy.

```
int *p1, int *p2;
```

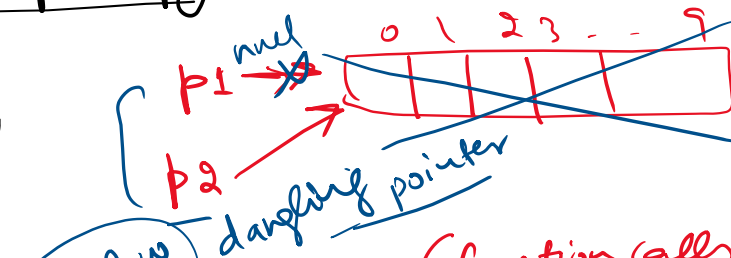
```
p1 = new int {10};
```

```
p2 = p1;
```

```
delete [] p1;
```

```
p1 = null ptr;
```

```
p2[5] = 10;
```



Shallow

Two or more pointers point to the same memory. (Address)

(function calls) by value

deep copy

pointer point to their own memory

① new memory for other pointer

② delete copy.